

HalftimeHash: Modern Hashing without 64-bit Multipliers or Finite Fields

Jim Apple^[0000–0002–8685–9451]

jim@jbapple.com

Abstract. HalftimeHash is a new algorithm for hashing long strings. The goals are few collisions (different inputs that produce identical output hash values) and high performance.

Compared to the fastest universal hash functions on long strings (clhash and UMASH), HalftimeHash decreases collision probability while also increasing performance by over 50%, exceeding 16 bytes per cycle.

In addition, HalftimeHash does not use any widening 64-bit multiplications or any finite field arithmetic that could limit its portability.

Keywords: Universal hashing · Randomized algorithms

1 Introduction

A hash family is a map from a set of seeds S and a domain D to a codomain C . A hash family H is called ε -almost universal (“ ε -AU” or just “AU”) when

$$\forall x, y \in D, x \neq y \implies \Pr_{s \in S}[H(s, x) = H(s, y)] \leq \varepsilon \in o(1)$$

The intuition behind this definition is that collisions can be made unlikely by picking randomly from a hash *family* independent of the input strings, rather than anchoring on a specific hash *function* such as MD5 that does not take a seed as an input. AU hash families are useful in hash tables, where collisions slow down operations and, in extreme cases, can turn linear algorithms into quadratic ones. [2,10,18,25]

HalftimeHash is a new “universe collapsing” hash family, designed to hash long strings into short ones. [1,11,21] This differs from short-input families like SipHash or tabulation hashing, which are suitable for hashing short strings to a codomain of 64 bits. [3,25] Universe collapsing families are especially useful for composition with short-input families: when n long strings are to be handled by a hash-based algorithm, a universe-collapsing family that reduces them to hash values of length $c \lg n$ bits for some suitable $c > 2$ produces zero collisions with probability $1 - O(n^{2-c})$. A short-input hash family can then treat the hashed values as if they were the original input values. [3,9,24,25] This technique applies not only to hash tables, but also to message-authentication codes, load balancing in distributed systems, privacy amplification, randomized geometric algorithms, Bloom filters, and randomness extractors. [4,9,12,13,22,23]

On strings longer than 1KB, HalftimeHash is typically 55% faster than clhash, the AU hash family that comes closest in performance.

HalftimeHash also has tunable output length and low probabilities of collision for applications that require them, such as one-time authentication.[5] The codomain has size 16, 24, 32, or 40 bytes, and ϵ varies depending on the codomain (see Figure 2 and Section 6).

1.1 Portability

In addition to high speed on long strings, HalftimeHash is designed for a simple implementation that is easily portable between programming languages and machine ISA’s. HalftimeHash uses less than 1200 lines of code in C++ and can take advantage of vector ISA extensions, including AVX-512, AVX2, SSE, and NEON.¹

Additionally, no multiplications from $\mathbb{Z}_{2^{64}} \times \mathbb{Z}_{2^{64}}$ to $\mathbb{Z}_{2^{128}}$ are needed. This is in support of two portability goals – the first is portability to platforms or programming languages without native widening unsigned 64-bit multiplications. Languages like Java, Python, and Swift can do these long multiplications, but not without calling out to C or slipping into arbitrary-precision-integer code. The other reason HalftimeHash avoids 64-bit multiplications is portability to SIMD ISA extensions, which generally do not contain widening 64-bit multiplication.

1.2 Prior Almost-Universal Families

There are a number of fast hash algorithms that run at rates exceeding 8 bytes per cycle on modern x86-64 processors, including Fast Positive Hash, falckhash, xxh, MeowHash, and UMASH, and clhash. [26] Of these, only clhash and UMASH include claims of being AU; each of these uses finite fields and the x86-64 instruction for carryless (polynomial) multiplication.

Rather than tree hashing, hash families like clhash and UMASH use polynomial hashing (based on Horner’s method) to hash variable-length strings down to fixed-size output. That approach requires 64-bit multiplication and also reduction modulo a prime (in $\mathbb{Z}$ or in $\mathbb{Z}_2[x]$), limiting its usability in SIMD ISA extensions.

1.3 Outline

The rest of this paper is organized as follows: Section 3 covers prior work that HalftimeHash builds upon. Section 4 introduces a new generalization of Nandi’s “Encode, Hash, Combine” algorithm.[20] Section 5 discusses specific implementation choices in HalftimeHash to increase performance. Section 6 analyzes and tests HalftimeHash’s performance.

¹ <https://github.com/jbapple/HalftimeHash>

2 Notations and Conventions

Input string length n is measured in 32-bit *words*. “32-bit multiplication” means multiplying two unsigned 32-bit words and producing a single 64-bit word. “64-bit multiplication” similarly refers to the operation producing a 128-bit product. All machine integers are unsigned.

Sequences are denoted by angled brackets: “ $\langle$ ”, “ $\rangle$ ”, and $\triangleleft$ prepends a value onto a sequence. Subscripts indicate a numbered component of a sequence, starting at 0. Contiguous half-open subsequences are denoted “ $x[y, z)$ ”, meaning $\langle x_y, x_{y+1}, \dots, x_{z-1} \rangle$.

$\perp$ is a new symbol not otherwise in the alphabet of words

In the definition of ε -almost universal, ε is called the *collision probability* of H ; it is inversely related to H ’s *output entropy*, $-\lg \varepsilon$. The seed is sometimes referred as *input entropy*, which is distinguished from the output entropy both because it is an explicit part of the input and because it is measured in words or bytes, not bits.

Each step of HalftimeHash applies various transforms to groups of input values. These groups are called *instances*. The processing of a transform on a single instance is called an *execution*.

Instances are logically contiguous but physically strided, for the purpose of simplifying SIMD processing. A physically contiguous set between two items in a single instance is called a *block*; the number of words in a block is called the *block size*. Because instances are logically contiguous, when possible, the analysis will elide references to the block size.

Tree hashing examples use a hash family parameter H that takes two words as input, but this can be easily extended to hash functions taking more than two words of input, in much the same way that binary trees are a special case of B-trees.

HalftimeHash produces output that is collision resistant among strings of the same length. Adding collision resistance between strings of *different* lengths to such a hash family requires only appending the length at the end of the output.

HalftimeHash variants will be specified by their number of output bytes: HalftimeHash16, HalftimeHash24, HalftimeHash32, or HalftimeHash40.

Except where otherwise mentioned, all benchmarks were run on an Intel i7-7800x (a Skylake X chip that supports AVX512), running Ubuntu 18.04, with Clang++ 11.0.1.

3 Prior Work

This section reviews hashing constructions that form components of HalftimeHash. In order to put these in context, a broad outline of HalftimeHash is in order.

HalftimeHash can be thought of as a tree-based, recursively-defined hash function. The leaves of the tree are the words of the unhashed input; the root is the output value. Every internal node has multiple inputs and a single output, corresponding with the child and parent nodes in the tree.

To a first approximation, a string is hashed by breaking it up into some number of contiguous parts, hashing each part, then combining those hash values. When the size of the input is low enough, rather than recurse, a construction called “Encode, Hash, Combine” (or “EHC”) is used to hash the input.

3.1 Tree Hash

HalftimeHash’s structure is based on a tree-like hash as described by Carter and Wegman. [8, Section 3] To hash a string, we use $\lceil \lg n \rceil$ randomly-selected keys k_i and a hash family H that hashes two words down to one. Then the tree hash T of a string $s[0, n)$ is defined recursively as:

$$\begin{aligned} T(k, \langle x \rangle) &\stackrel{\text{def}}{=} x \\ T(k, s[0, n)) &\stackrel{\text{def}}{=} H(k_{\lceil \lg n - 1 \rceil}, T(k, s[0, 2^{\lceil \lg n - 1 \rceil})), T(k, s[2^{\lceil \lg n - 1 \rceil}, n))) \end{aligned} \quad (1)$$

Carter and Wegman show that if H is ε -AU, T is $m\varepsilon$ -AU for input that has length exactly 2^m . Later, Boesgaard et al. extended this proof to strings with lengths that are not a power of two.[7]

3.2 NH

In HalftimeHash, NH, an almost-universal hash family, is used at the nodes of tree hash to hash small, fixed-length sequences:[6]

$$\sum_{i=0}^m (d_{2i} + s_{2i})(d_{2i+1} + s_{2i+1})$$

where $d, s \in \mathbb{Z}_{2^{32}}^{2^{m+2}}$ are the input string and the input entropy, respectively. The $d_j + s_j$ additions are in the ring $\mathbb{Z}_{2^{32}}$, while all other operations are in the ring $\mathbb{Z}_{2^{64}}$. NH is 2^{-32} -AU. In fact, it satisfies a stronger property, 2^{-32} -A Δ U:[6]

Definition 1. A hash family H is said to be ε -almost Δ -universal (or just A Δ U) when

$$\forall x, y, \delta, \Pr_s[H(s, x) - H(s, y) = \delta] \leq \varepsilon \in o(1)$$

In tree nodes (though not in EHC, covered below), a variant of NH is used in which the last input pair is not hashed, thereby increasing performance:

$$\left(\sum_{i=0}^{m-1} (d_{2i} + s_{2i})(d_{2i+1} + s_{2i+1}) \right) + d_{2m} + 2^{32}d_{2m+1}$$

This hash family is still 2^{-32} -AU.[7]

3.3 Encode, Hash, Combine

At the leaves of the tree hash, HalftimeHash uses the “Encode, Hash, Combine” algorithm.[20] EHC is parameterized by an erasure code with “minimum distance” k , which is a map on sequences of words such that any two input values that differ in *any* location produce encoded outputs that differ in *at least* $k > 1$ locations after encoding.

The EHC algorithm is:

1. A sequence of words is processed by an erasure code with minimum distance k , producing a longer encoded sequence.
2. Each word in the encoded sequence is hashed using an A Δ U family with independently and randomly chosen input entropy.
3. A linear transformation T is applied to the resulting sequence of hash values. The codomain of T has dimension k , and T must have the property that any k columns of it are linearly independent.

Nandi proved that if the EHC matrix product is over a finite field, EHC is ε^k -AU. This AU collision probability could be achieved on the same input by instead running k copies of NH, but that would perform mk multiplications to hash m words, while EHC requires $m + k$ multiplications, excluding the multiplications implicit in applying T . That exclusion is the topic of Section 4.

4 Generalized EHC

At first glance, EHC might not look like it will reduce the number of multiplications needed, as the application of linear transformations usually requires multiplication. However, since T is not part of the randomness of the hash family, it can be designed to contain only values that are trivial to multiply by, such as powers of 2.

The constraint in [20] requires that any k columns of T form an invertible matrix. This is not feasible in linear transformations on $\mathbb{Z}_{2^{64}}$ in most useful dimensions. For instance, in HalftimeHash24, a 3×9 matrix T is used. Any such matrix will have at least one set of three columns with an even determinant, and which therefore has a non-trivial kernel.

Proof. Let U be a matrix over $\mathbb{Z}_2$ formed by reducing each entry of T modulo 2. Then $(\det T) \bmod 2 \equiv \det U$. Since there are only 7 unique non-zero columns of size 3 over $\mathbb{Z}_2$, by the pigeonhole principle, some two columns x, y of U must be equal. Any set of columns that includes both x and y has a determinant of 0 mod 2. $\square$

Let k be the minimum distance of the erasure code. While Nandi proved that EHC is ε^k -AU over a finite field, $\mathbb{Z}_{2^{64}}$ is not a finite field. However, there are similarities to a finite field, in that there are some elements in $\mathbb{Z}_{2^{64}}$ with inverses. Some other elements in $\mathbb{Z}_{2^{64}}$ are zero divisors, but only have one value that they can be multiplied by to produce 0. A variant of Nandi’s proof is presented here as a warm-up to explain the similarities. [20]

Lemma 1. *When the matrix product is taken over a field, if the hash function H used in step 2 is ε -A Δ U, EHC is ε^k -A Δ U.*

Proof. Let $\bar{H}$ be defined as $\bar{H}(s, x)_i \stackrel{\text{def}}{=} H(s_i, x_i)$. Let J be the encoding function that acts on x and y , producing an encoding of length e . Given that x and y differ, let F be k locations where $J(x)_i \neq J(y)_i$. Let $T|_F$ be the matrix formed by the columns of T where the column index is in F and let $\bar{H}|_F$ similarly be $\bar{H}$ restricted to the indices in F . Conditioning over the $e - k$ indices not in F , we want to bound

$$\Pr_s[T|_F \bar{H}|_F(s, J(x)) - T|_F \bar{H}|_F(s, J(y)) = \delta] \quad (2)$$

Since any k columns of T are independent, $T|_F$ is non-singular, and the equation is equivalent to $\bar{H}|_F(s, J(x)) - \bar{H}|_F(s, J(y)) = T|_F^{-1} \delta$, which implies

$$\bigwedge_{i \in F} H(s_i, J(x)_i) - H(s_i, J(y)_i) = \beta_i$$

where $\beta \stackrel{\text{def}}{=} T|_F^{-1} \delta$.

Since the s_i are all chosen independently, the probability of the conjunction is the product of the probabilities, showing

$$\begin{aligned} & \Pr_s[T|_F \bar{H}|_F(s, J(x)) - T|_F \bar{H}|_F(s, J(y)) = \delta] \\ & \leq \prod_{i \in F} \Pr_s[H(s_i, J(x)_i) - H(s_i, J(y)_i) = \beta_i] \end{aligned}$$

and since H is A Δ U, this probability is ε^k . $\square$

Note that this lemma depends on k being the minimum distance of the code. If the distance were less than k , then the matrix would be smaller, increasing the probability of collisions.

In the non-field ring $\mathbb{Z}_{2^{64}}$, the situation is altered. “Good” matrices are those in which the determinant of any k columns is divisible only by a small power of two. The intuition is that, since matrices in $\mathbb{Z}_{2^{64}}$ with odd determinants are invertible, the “closer” a determinant is to odd (meaning it is not divisible by large powers of two), the “closer” it is to invertible.

Theorem 1. *Let p be the largest power of 2 that divides the determinant of any k columns in T . The EHC step of HalftimeHash is $2^{k(p-32)}$ -A Δ U when using NH as the hash family.*

Proof. In HalftimeHash, the proof of the lemma above unravels at the reliance upon the trivial kernel of $T|_F$. The columns of T in HalftimeHash are linearly independent, so the matrix $T|_F$ is injective in rings without zero dividers, but not necessarily injective in $\mathbb{Z}_{2^{64}}$.

However, even in $\mathbb{Z}_{2^{64}}$, the adjugate matrix $\text{adj}(A)$ has the property that $A \cdot \text{adj}(A) = \text{adj}(A) \cdot A = \det(A)I$. Let $\det(T|_F) = q2^{p'}$, where q is odd and

$p' \leq p$. Now (2) reduces to

$$\begin{aligned}
& \Pr_s[T|_F \bar{H}|_F(s, x) - T|_F \bar{H}|_F(s, y) = \delta] \\
& \leq \Pr_s[\text{adj}(T|_F)T|_F \bar{H}|_F(s, x) - \text{adj}(T|_F)T|_F \bar{H}|_F(s, y) = \text{adj}(T|_F)\delta] \\
& = \Pr_s[q^{2^{p'}} \bar{H}|_F(s, x) - q^{2^{p'}} \bar{H}|_F(s, y) = \text{adj}(T|_F)\delta] \\
& = \Pr_s[2^{p'} \bar{H}|_F(s, x) - 2^{p'} \bar{H}|_F(s, y) = q^{-1} \text{adj}(T|_F)\delta]
\end{aligned}$$

Now letting $\beta = q^{-1} \text{adj}(T|_F)\delta$ and letting the modulo operator extend pointwise to vectors, we have

$$\begin{aligned}
& = \Pr_s[\bar{H}|_F(s, x) - \bar{H}|_F(s, y) \equiv \beta \bmod 2^{64-p'}] \\
& = \Pr_s\left[\bigwedge_{i \in F} H(s_i, x_i) - H(s_i, y_i) \equiv \beta_i \bmod 2^{64-p'}\right] \\
& = \prod_{i \in F} \Pr_s\left[H(s_i, x_i) - H(s_i, y_i) \equiv \beta_i \bmod 2^{64-p'}\right] \\
& = \left(2^{p'} 2^{-32}\right)^{|F|} = 2^{k(p'-32)}
\end{aligned}$$

This quantity is highest when p' is at its maximum over all potential sets of columns F , and p' is at most p , by the definition of p . $\square$

This generalized version of EHC is used in the implementation of HalftimeHash described in Section 5, with $p \leq 2^3$.

5 Implementation

This section describes the specific implementation choices made in HalftimeHash to ensure high output entropy and high performance. The algorithm performs the following steps:

- Generalized EHC on instances of the unhashed input, producing 2, 3, 4, or 5 output words (of 64 bits each) per input instance
- 2, 3, 4, or 5 exexutions of tree hash (with independently and randomly chosen input entropy) on the output of EHC, with NH at each internal node, producing a sequence of words logarithmic in the length of the input string, as described below in Equation 3
- NH on the output of each tree hash, producing 16, 24, 32, or 40 bytes

5.1 EHC

In addition to the trivial distance-2 erasure code of XOR'ing the words together and appending that as an additional word, HalftimeHash uses non-linear erasure codes discovered by Gabrielyan with minimum distance 3, 4, or 5. [14,15,16] These codes can be computed without any multiplications.

For the linear transformations, HalftimeHash uses matrices T selected so that the largest power of 2 that divides any determinant is 2^2 or 2^3 . For instance, for the HalftimeHash24 variant, T has a p of 2^2 :

$$\begin{pmatrix} 0 & 0 & 1 & 4 & 1 & 1 & 2 & 2 & 1 \\ 1 & 1 & 0 & 0 & 1 & 4 & 1 & 2 & 2 \\ 1 & 4 & 1 & 1 & 0 & 0 & 2 & 1 & 2 \end{pmatrix}$$

For other output widths, HalftimeHash uses

	HalftimeHash16	HalftimeHash32	HalftimeHash40
T	$\begin{pmatrix} 1 & 0 & 1 & 1 & 2 & 1 & 4 \\ 0 & 1 & 1 & 2 & 1 & 4 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 4 & 2 & 4 & 1 & 1 \\ 0 & 1 & 2 & 0 & 0 & 1 & 1 & 2 & 4 & 1 \\ 2 & 0 & 1 & 0 & 4 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 & 4 & 1 & 2 & 8 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 & 2 & 4 \\ 0 & 1 & 0 & 0 & 0 & 1 & 2 & 1 & 7 \\ 0 & 0 & 1 & 0 & 0 & 1 & 3 & 8 & 5 \\ 0 & 0 & 0 & 1 & 0 & 1 & 4 & 9 & 8 \\ 0 & 0 & 0 & 0 & 1 & 1 & 5 & 3 & 9 \end{pmatrix}$
p	2^2	2^3	2^3

The input group lengths for the EHC input are 6, 7, 7, and 5, as can be seen from the dimensions of the matrices: columns + 1 – rows. Note that each of these matrices contains coefficients that can be multiplied by with no more than two shifts and one addition.

5.2 Tree Hash

For the tree hashing at internal nodes (above the leaf nodes, which use EHC), $k \in \{2, 3, 4, 5\}$ tree hashes are executed with independently-chosen input entropy, producing output entropy of $-k \lg \varepsilon$. From the result from Carter and Wegman on the entropy of tree hash of a tree of height m , the resulting hash function is $m\varepsilon^k$ -AU.

The key lemma they need is that almost universality is composable:

Lemma 2 (Carter and Wegman). *If F is ε_F -AU, G is ε_G -AU, then*

- $F \circ G$ where $F \circ G(\langle k_F, k_G \rangle, x) \stackrel{\text{def}}{=} F(k_F, G(k_G, x))$ is $(\varepsilon_F + \varepsilon_G)$ -AU.
- $\langle F, G \rangle$ where $\langle F, G \rangle(\langle k_F, k_G \rangle, \langle x, y \rangle) \stackrel{\text{def}}{=} \langle F(k_F, x), G(k_G, y) \rangle$, is $\max(\varepsilon_F, \varepsilon_G)$ -AU, even if $F = G$ and $k_F = k_G$.

The approach in Badger of using Equation 1 to handle words that are not in perfect trees can be increased in speed with the following method: For HalftimeHash, define $\hat{T}$ as a family taking as input sequences of any length n and producing sequences of length $\lceil \lg n \rceil$ as follows, using Carter and Wegman's T defined in Section 3:

$$\begin{aligned} \hat{T}_0(k, \langle \rangle) &\stackrel{\text{def}}{=} \langle \perp \rangle \\ \hat{T}_0(k, \langle x \rangle) &\stackrel{\text{def}}{=} \langle x \rangle \\ \hat{T}_{i+1}(k, s[0, n]) &\stackrel{\text{def}}{=} \begin{cases} \perp \triangleleft \hat{T}_i(k, s[0, n]) & \text{if } 2^i > n \\ T(k, s[0, 2^i]) \triangleleft \hat{T}_i(k, s[2^i, n]) & \text{if } 2^i \leq n \end{cases} \end{aligned} \quad (3)$$

There is one execution of T for every 1 in the binary representation of n . By an induction on $\lceil \lg n \rceil$ using the composition lemma, $\hat{T}$ is $\varepsilon \lceil \lg n \rceil$ -AU.

The output of $\hat{T}$ is then hashed using an NH instance of size $\lceil \lg n \rceil$. This differs from Badger, where T is used to fully consume the input without the use of additional input entropy; T produces a single word per execution, while $\hat{T}$ needs to be paired with NH post-processing in order to achieve that.[7] Empirically, $\hat{T}$ has better performance than the Badger approach.

6 Performance

This section tests and analyzes HalftimeHash performance, including an analysis of the output entropy.

6.1 Analysis

The parameters used in this analysis are:

- b the number of 64-bit words in a block. Blocks are used to take advantage of SIMD units.
- d is the number of elements in each EHC instance before applying the encoding.
- e is the number of blocks in EHC after applying the encoding.
- f is fanout, the width of the NH instance at tree hash nodes.
- k is the number of blocks produced by the Combine step of EHC. This is also the minimum distance of the erasure code, as described above.
- p is the maximum power of 2 that divides a determinant of any $k \times k$ matrix made from columns of the matrix T ; doubling p increases ε by a factor of 2^k .
- w is the number of blocks in each item used in the Encode step of EHC.

In HalftimeHash24,

$$(b, d, e, f, k, p, w) = (8, 7, 9, 8, 3, 2^2, 3)$$

Each EHC execution reads in dw blocks, produces e blocks, uses ew words of input entropy, and performs ew multiplications.

For the tree hash portion of HalftimeHash, the height of the k trees drives multiple metrics. Each tree has $\lfloor n/bdw \rfloor$ blocks as input and every level execution forms a complete f -ary execution tree. The height of the tree is thus $h \stackrel{\text{def}}{=} \lceil \log_f \lfloor n/bdw \rfloor \rceil$.

Lemma 3. *The tree hash is $2^{k \lg h - 32k}$ -AU.*

Proof. Carter and Wegman showed that tree hash has collision probability of $h\varepsilon$, where ε is the collision probability of a single node. Each tree node uses NH, so a single tree has collision probability $h2^{-32}$. A collision occurs for HalftimeHash at the tree hash stage if and only if all k trees collide, which has probability $(h2^{-32})^k$, assuming that the EHC step didn't already induce a collision. $\square$

The amount of input entropy needed is proportional to the height of the tree, with $f - 1$ words needed for every level. HalftimeHash uses different input entropy for the k different trees, so the total number of 64-bit words of input entropy used in the tree hash step is $(f - 1)hk$.

The number of multiplications performed is identical to the number of 64-bit words input, $kb\lfloor n/bdw \rfloor$.

The result of the tree hash is processed through NH, which uses $bfhk$ words of entropy and just as many multiplications.

There can also be as much as bdw words of data in the raw input that are not read by HalftimeHash, as they are less than the input size of one instance of EHC. Again, NH is used on this data, but now hashing k times, since this data has not gone through EHC. That requires $bdwk$ words of entropy and just as many multiplications.

For this previously-unread data, the number of words of entropy needed can be reduced by nearly a factor of k using the Toeplitz construction. Let r be the sequence of random words used to hash it. Instead of using $r[i, bdw, (i + 1)bdw]$ as the keys to hash component i with, HalftimeHash uses $r[i, bdw + i]$. This construction for multi-part hash output is $A\Delta U$. [20,27]

6.2 Cumulative Analysis

The combined collision probability is $2^{-32k} (2^{kp} + h^k + 1)$. For HalftimeHash24, and for strings less than an exabyte in length, this is more than 83 bits of entropy.

The combined input entropy needed (in words) is $ew + (f - 1)hk + bfhk + bdw + k - 1$. HalftimeHash24 requires 8.4KB input entropy for strings of length up to one megabyte and 34KB entropy for strings of length up to one exabyte.

The number of multiplications is dominated by the EHC step, since the total is $(ew + k)b\lfloor n/bdw \rfloor + O(\log n)$ and ew is significantly larger than k . For a string of length 1MB, 84% of the multiplications happen in the EHC step. Intel's VTune tool show the same thing: 86% of the clock cycles are spent in the EHC step. ²

6.3 Benchmarks

HalftimeHash passes all correctness and randomness tests in the SMHasher test suite; for a performance comparison, see Figure 1 and [26].

Figure 2 displays the relationship between output entropy and throughput for HalftimeHash, UMASH, and clhash.³ Adding more output entropy increases the number of non-linear arithmetic operations that any hash function has to perform.[20] The avoidance of doubling the number of multiplications for twice the output size is one of the primary reasons that HalftimeHash24, -32, and -40 are faster than running clhash or UMASH with 128-bit output. (The other is that carryless multiplication is not supported as a SIMD instruction.)

² Similarly, clhash and UMASH, which are based on 64-bit carryless NH, have their execution times dominated by the multiplications in their base step. [17,19]

³ UMASH and clhash are the fastest AU families for string hashing

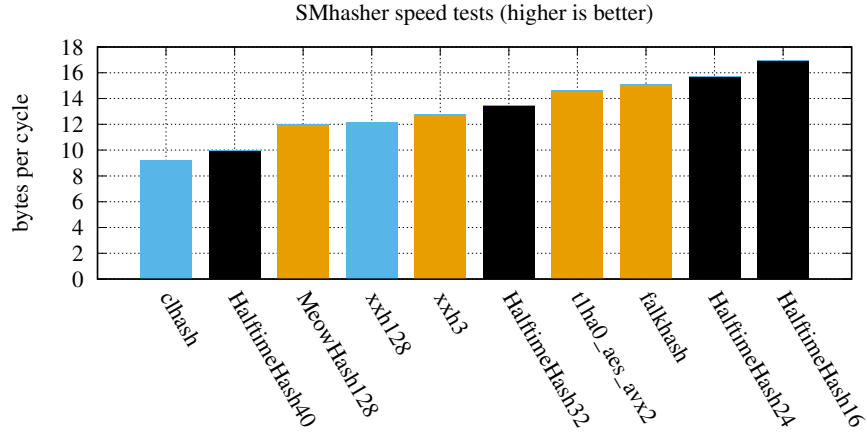


Fig. 1. The two fastest variants of HalftimeHash are faster than all hash families in the SMHasher suite on 256KiB strings on an i7-7800x, even families that come with no AU guarantees. [26] Of the families here, only HalftimeHash and xxh128 pass all SMHasher tests, and only HalftimeHash and clhash are AU.

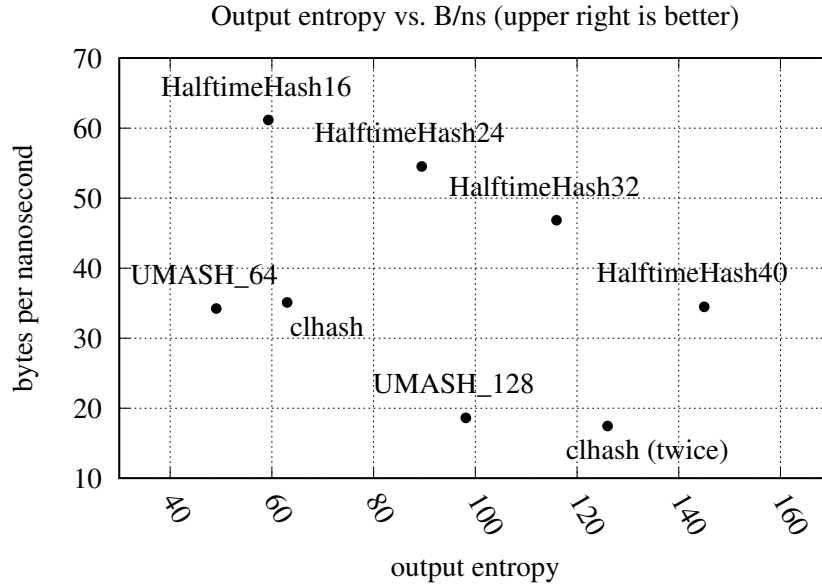


Fig. 2. Trade-offs for almost-universal string hashing functions on strings of size 250KB on an i7-7800x. UMASH comes in two variants based on the output width in bits; clhash doesn't, but running clhash twice is included in the chart. For each clhash / UMASH version, at least one version of HalftimeHash is faster and has lower collision probability.

Figure 3 adds comparisons between clhash, UMASH, and HalftimeHash across input sizes and processor manufacturers. Although these two machines support different ISA vector extensions, the pattern is similar: for large enough input, HalftimeHash’s throughput exceeds that of the carryless multiplication families.

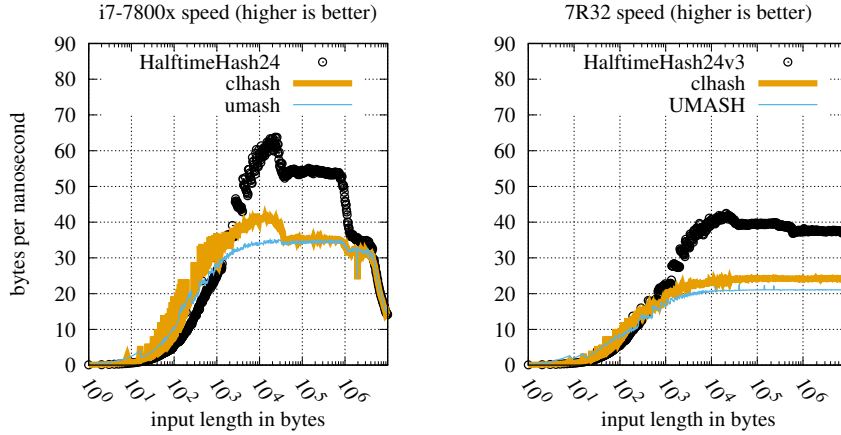


Fig. 3. Comparison of Intel (i7-7800x) and AMD (EC2 c5a.large, 7R32) performance. On both chips HalftimeHash24 is faster than clhash and UMASH for long strings. The “v3” after the name of the AMD HalftimeHash indicates block size: v3 means a 256-bit block size, while v4 (the default) means 512-bit block size. AMD chips do not support AVX-512, but still HalftimeHash with 256-bit blocks exceeds the speed of cmul-based hashing methods by up to a factor of 2.

7 Future Work

Areas of future research include:

- Tuning for JavaScript, which has no native 32-bit multiplication support, as it natively supports only double-precision floating point numbers
- Comparisons against hash algorithms in the Linux kernel, including Poly1305 and `crc32_pclmul_1e_16`
- Benchmarks on POWER and ARM ISA’s
- EHC benchmarks using 64-bit multiplication – carryless or integral

Acknowledgments Thanks to Daniel Lemire, Paul Khuong, and Guy Even for helpful discussions and feedback.

References

1. Alon, N., Dietzfelbinger, M., Miltersen, P.B., Petrank, E., Tardos, G.: Linear hash functions. *J. ACM* **46**(5), 667–683 (Sep 1999), <https://doi.org/10.1145/324133.324179>
2. Apple, J.: Ensure monotonic count(distinct x) performance (2015), <https://issues.apache.org/jira/browse/IMPALA-2653>, accessed on 2020-12-32
3. Aumasson, J.P., Bernstein, D.J.: SipHash: a fast short-input PRF. In: International Conference on Cryptology in India. pp. 489–508. Springer (2012)
4. Bernstein, D.J.: The Poly1305-AES message-authentication code. In: International Workshop on Fast Software Encryption. pp. 32–49. Springer (2005)
5. Bernstein, D.J.: Cryptography in NaCl. *Networking and Cryptography library* **3**, 385 (2009)
6. Black, J., Halevi, S., Krawczyk, H., Krovetz, T., Rogaway, P.: UMAC: Fast and secure message authentication. In: Annual International Cryptology Conference. pp. 216–233. Springer (1999), https://link.springer.com/chapter/10.1007/3-540-48405-1_14
7. Boesgaard, M., Christensen, T., Zenner, E.: Badger – a fast and provably secure mac. In: Ioannidis, J., Keromytis, A., Yung, M. (eds.) *Applied Cryptography and Network Security*. pp. 176–191. Springer Berlin Heidelberg, Berlin, Heidelberg (2005)
8. Carter, J.L., Wegman, M.N.: Universal classes of hash functions. *Journal of Computer and System Sciences* **18**(2), 143 – 154 (1979), <http://www.sciencedirect.com/science/article/pii/0022000079900448>
9. Chung, K.M., Mitzenmacher, M., Vadhan, S.: Why simple hash functions work: Exploiting the entropy in a data stream. *Theory of Computing* **9**(30), 897–945 (2013), <http://www.theoryofcomputing.org/articles/v009a030>
10. Crosby, S.A., Wallach, D.S.: Denial of service via algorithmic complexity attacks. In: *Proceedings of the 12th USENIX Security Symposium*. pp. 29–44 (2003)
11. Dietzfelbinger, M.: Universal Hashing via Integer Arithmetic Without Primes, Revisited, pp. 257–279. Springer International Publishing (2018), https://doi.org/10.1007/978-3-319-98355-4_15
12. Dietzfelbinger, M., Hagerup, T., Katajainen, J., Penttonen, M.: A reliable randomized algorithm for the closest-pair problem. *Journal of Algorithms* **25**(1), 19–51 (1997)
13. Dodis, Y., Ostrovsky, R., Reyzin, L., Smith, A.: Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. *SIAM Journal on Computing* **38**(1), 97–139 (Jan 2008), <http://dx.doi.org/10.1137/060651380>
14. Gabrielyan, E.: Erasure resilient (10,7) code (2005), <https://docs.switzzernet.com/people/emin-gabrielyan/051102-erasure-10-7-resilient/>, accessed: 2020-11-26
15. Gabrielyan, E.: Erasure resilient MDS code with four redundant packets (2005), <https://docs.switzzernet.com/people/emin-gabrielyan/051103-erasure-9-5-resilient/>, accessed: 2020-11-26
16. Gabrielyan, E.: Erasure resilient (9,7)-code (2005), <https://docs.switzzernet.com/people/emin-gabrielyan/051101-erasure-9-7-resilient/>, accessed: 2020-11-26
17. Khuong, P.: UMASH: a fast and universal enough hash (2020), <https://engineering.backtrace.io/2020-08-24-umash-fast-enough-almost-universal-fingerprinting/>
18. Landau, J.: Exposure of HashMap iteration order allows for $O(n^2)$ blowup (2016), <https://github.com/rust-lang/rust/issues/36481>, accessed on 2020-12-32

19. Lemire, D., Kaser, O.: Faster 64-bit universal hashing using carry-less multiplications. *Journal of Cryptographic Engineering* **6**(3), 171–185 (2016)
20. Nandi, M.: On the minimum number of multiplications necessary for universal hash functions. In: Cid, C., Rechberger, C. (eds.) *Fast Software Encryption*. pp. 489–508. Springer Berlin Heidelberg, Berlin, Heidelberg (2015)
21. Pagh, R., Rodler, F.F.: Cuckoo hashing. *Journal of Algorithms* **51**(2), 122 – 144 (2004), <http://www.sciencedirect.com/science/article/pii/S0196677403001925>
22. Renner, R., König, R.: Universally composable privacy amplification against quantum adversaries. In: *Theory of Cryptography Conference*. pp. 407–425. Springer (2005)
23. Stoica, I., Morris, R., Karger, D., Kaashoek, M.F., Balakrishnan, H.: Chord: A scalable peer-to-peer lookup service for internet applications. *ACM SIGCOMM Computer Communication Review* **31**(4), 149–160 (2001)
24. Thorup, M.: String Hashing for Linear Probing, pp. 655–664. Society for Industrial and Applied Mathematics (2009), <https://epubs.siam.org/doi/abs/10.1137/1.9781611973068.72>
25. Thorup, M.: Fast and powerful hashing using tabulation. *CoRR* **abs/1505.01523** (2017), <http://arxiv.org/abs/1505.01523>
26. Urban, R., et al.: Smhasher (2020), <https://github.com/rurban/smhasher>
27. Woelfel, P.: Efficient strongly universal and optimally universal hashing. In: *Mathematical Foundations of Computer Science 1999, 24th International Symposium*. LNCS, vol. 1672, pp. 262–272 (1999), https://doi.org/10.1007/3-540-48340-3_24